Game Theory-Assignment3

Xinyi Yang

February 1st 2026

Exercise 6

(a) Saddle-point conditions

The quadratic form is given by

$$J(u,d,x) = \begin{bmatrix} u^\top & d^\top & x^\top \end{bmatrix} \begin{bmatrix} P_{uu} & P_{ud} & P_{ux} \\ P_{ud}^\top & P_{dd} & P_{dx} \\ P_{ux}^\top & P_{dx}^\top & P_{xx} \end{bmatrix} \begin{bmatrix} u \\ d \\ x \end{bmatrix}.$$

Expanding the quadratic form:

$$J = u^\top P_{uu} u + 2u^\top P_{ud} d + 2u^\top P_{ux} x + d^\top P_{dd} d + 2d^\top P_{dx} x + x^\top P_{xx} x.$$

A saddle point (u^*, d^*) must satisfy the first-order conditions:

$$\frac{\partial J}{\partial u} = 0, \quad \frac{\partial J}{\partial d} = 0.$$

Computing the gradients:

$$\frac{\partial J}{\partial u} = 2(P_{uu} u + P_{ud} d + P_{ux} x) = 0,$$
$$\frac{\partial J}{\partial d} = 2(P_{ud}^\top u + P_{dd} d + P_{dx} x) = 0.$$

Dividing by 2 and writing in matrix form:

$$\begin{bmatrix} P_{uu} & P_{ud} \\ P_{ud}^\top & P_{dd} \end{bmatrix} \begin{bmatrix} u \\ d \end{bmatrix} = - \begin{bmatrix} P_{ux} \\ P_{dx} \end{bmatrix} x.$$

Since P_{uu} is symmetric positive definite and P_{dd} is symmetric negative definite, the block matrix is invertible (by the hint, the Schur complement $P_{dd} - P_{ud}^\top P_{uu}^{-1} P_{ud}$ is negative definite, hence invertible). Thus,

$$\begin{bmatrix} u^* \\ d^* \end{bmatrix} = - \begin{bmatrix} P_{uu} & P_{ud} \\ P_{ud}^\top & P_{dd} \end{bmatrix}^{-1} \begin{bmatrix} P_{ux} \\ P_{dx} \end{bmatrix} x.$$

This is the unique solution to the gradient conditions.

(b) Value of the game

Substitute (u^*, d^*) back into $J(u,d,x)$. Let

$$M = \begin{bmatrix} P_{uu} & P_{ud} \\ P_{ud}^\top & P_{dd} \end{bmatrix}, \quad N = \begin{bmatrix} P_{ux} \\ P_{dx} \end{bmatrix}.$$

Then $[u^* \ d^*]^\top = -M^{-1} N x$.

The value of the game is:

$$\begin{aligned} J(u^*, d^*, x) &= x^\top P_{xx} x + 2 \begin{bmatrix} u^* \\ d^* \end{bmatrix}^\top N x + \begin{bmatrix} u^* \\ d^* \end{bmatrix}^\top M \begin{bmatrix} u^* \\ d^* \end{bmatrix} \\ &= x^\top P_{xx} x + 2(-M^{-1}Nx)^\top N x + (-M^{-1}Nx)^\top M(-M^{-1}Nx) \\ &= x^\top P_{xx} x - 2x^\top N^\top M^{-1} N x + x^\top N^\top M^{-1} M M^{-1} N x \\ &= x^\top P_{xx} x - 2x^\top N^\top M^{-1} N x + x^\top N^\top M^{-1} N x \\ &= x^\top \big(P_{xx} - N^\top M^{-1} N\big) x. \end{aligned}$$

Thus,

$$J(u^*, d^*, x) = x^\top \left(P_{xx} - \begin{bmatrix} P_{ux}^\top & P_{dx}^\top \end{bmatrix} \begin{bmatrix} P_{uu} & P_{ud} \\ P_{ud}^\top & P_{dd} \end{bmatrix}^{-1} \begin{bmatrix} P_{ux} \\ P_{dx} \end{bmatrix} \right) x.$$

This shows that (u^*, d^*) is a saddle-point equilibrium and gives the value of the game.

Exercise 7

Consider the zero-sum game with Player 1 mixing between two strategies with probability $p \in [0, 1]$. The expected payoffs against Player 2's two pure strategies are given by

$$E_1(p) = 2 - 2p, \qquad E_2(p) = 2 + 4p.$$

Player 2 aims to maximize the payoff, while Player 1 aims to minimize it. Therefore, the relevant objective for Player 1 is

$$\min_{p \in [0,1]} \max\{E_1(p), E_2(p)\}.$$

For all $p > 0$, we have $E_2(p) > E_1(p)$, which implies that

$$\max\{E_1(p), E_2(p)\} = E_2(p).$$

Since $E_2(p)$ is strictly increasing in p, the minimum of the upper envelope is attained at $p = 0$.

Thus, the optimal strategy for Player 1 is the pure strategy corresponding to $p = 0$, which guarantees the security value

$$V = E_2(0) = 2.$$

This game admits a *pure saddle-point*, and no non-trivial mixed strategy is required. The security equilibrium consists of Player 1 choosing the second row and Player 2 choosing the first column, yielding a value of 2.

Exercise 8

Consider the following two zero-sum matrix games, where Player 1 (row player) is the minimizer and Player 2 (column player) is the maximizer:

$$A = \begin{pmatrix} 2 & 6 & -2 & 10 \\ -6 & -4 & -3 & -6 \\ 0 & 4 & -3 & -8 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 & 1 & 2 & 3 \\ 1 & 0 & 1 & 2 \\ 0 & 1 & 0 & 1 \\ -1 & 0 & 1 & 0 \end{pmatrix}.$$

For each game, the mixed security level of Player 1 can be computed by solving the following linear program:

$$\begin{aligned} \min_{p, v} \quad & v \\ \text{subject to} \quad & M^\top p \le v \mathbf{1}, \\ & \mathbf{1}^\top p = 1, \\ & p \ge 0, \end{aligned}$$

where M denotes the corresponding payoff matrix (A or B), p is Player 1's mixed security policy, and v is the average security level.

The above linear programs are solved numerically using `linprog` in MATLAB. The resulting mixed strategies and security values guarantee that the expected payoff against any pure strategy of Player 2 does not exceed the corresponding average security level.

MATLAB Implementation

```
A = [ 2  6 -2 10;
     -6 -4 -3 -6;
      0  4 -3 -8 ];

B = [ 0  1  2  3;
      1  0  1  2;
      0  1  0  1;
     -1  0  1  0 ];

games = {A, B};
names = {'Game A', 'Game B'};

options = optimoptions('linprog', 'Display', 'off');

for k = 1:length(games)
    M = games{k};
    [m, n] = size(M);

    f = [zeros(m, 1); 1];
    Aineq = [M', -ones(n, 1)];
    bineq = zeros(n, 1);
    Aeq = [ones(1, m), 0];
    beq = 1;
    lb = [zeros(m, 1); -Inf];

    x = linprog(f, Aineq, bineq, Aeq, beq, lb, [], [], options);

    p_star = x(1:m);
    v_star = x(end);

    fprintf('\n=== %s ===\n', names{k});
    fprintf('Mixed security value V: %.4f\n', v_star);
    fprintf('Mixed security policy p*:\n');
    for i = 1:m
        fprintf('  Row %d: %.4f\n', i, p_star(i));
    end
end
```

Listing 1: MATLAB code for Exercise 8

Results

Solving the linear programs numerically using MATLAB yields the following results.

Game A. The mixed security value is

$$V_A = -3.0.$$

The corresponding mixed security policy for Player 1 is

$$p_A = (0,\ 0.875,\ 0.125).$$

Game B. The mixed security value is

$$V_B = 0.5.$$

The corresponding mixed security policy for Player 1 is

$$p_B = (0,\ 0,\ 0.5,\ 0.5).$$

Analysis

For both zero-sum matrix games, the mixed security levels and the corresponding mixed security policies are obtained by solving the security linear programs for Player 1.

In Game A, the optimal mixed strategy assigns positive probability only to the second and third rows, while the first row receives zero probability. This indicates that the first strategy is not required to guarantee the security level and is dominated in the minimax sense. The mixed security value $V_A = -3.0$ represents the worst-case expected payoff that Player 1 can ensure against all strategies of Player 2.

In Game B, the optimal mixed strategy assigns equal probability to the third and fourth rows, with zero probability on the first two rows. This symmetric allocation reflects that these two strategies are sufficient to achieve the security level. The mixed security value $V_B = 0.5$ is the guaranteed average payoff for Player 1 in the worst-case scenario.

In both games, the mixed security policies ensure that Player 1 minimizes the maximum expected payoff imposed by Player 2, and the corresponding values represent the average security levels of the games.

Exercise 9

Problem Statement

A circuit designer chooses a nominal resistor value from:

$$R_{\text{nom}} \in \{0.70, 0.75, 0.80, \ldots, 1.20\} \quad \text{(11 values, 0.05 spacing)}$$

An opponent selects a percentage deviation from:

$$\delta \in \{-10\%, -9\%, \ldots, 10\%\} \quad \text{(21 values, 1\% spacing)}$$

The cost function is:

$$J(R_{\text{nom}}, \delta) = \left|1 - \frac{1}{R}\right|, \quad R = R_{\text{nom}}(1+\delta)$$

where the designer (P1) minimizes and the opponent (P2) maximizes.

Game Matrix Construction

The game is represented by an 11×21 matrix A where:

$$A_{ij} = \left|1 - \frac{1}{R_{\text{nom}}^{(i)}(1+\delta^{(j)})}\right|, \quad i = 1, \ldots, 11,\ j = 1, \ldots, 21$$

MATLAB Implementation

```
R_nom = 0.70:0.05:1.20;
delta = -0.10:0.01:0.10;

m = length(R_nom);
n = length(delta);

A = zeros(m, n);
for i = 1:m
    for j = 1:n
```

```
        R = R_nom(i) * (1 + delta(j));
        A(i, j) = abs(1 - 1/R);
    end
end

max_per_row = max(A, [], 2);
[V_pure, best_row_idx] = min(max_per_row);

min_per_col = min(A, [], 1);
[maximin_val, best_col_idx] = max(min_per_col);

fprintf('Matrix size: %d (R_nom choices)    %d (delta choices)\n', m, n);
fprintf('\n--- Pure Strategy Analysis ---\n');
fprintf('Designer (P1, minimizer):\n');
fprintf('  Security level (minimax): %.6f\n', V_pure);
fprintf('  Optimal pure strategy: R_nom = %.2f   \n', R_nom(best_row_idx));
fprintf('\nOpponent (P2, maximizer):\n');
fprintf('  Security level (maximin): %.6f\n', maximin_val);
fprintf('  Optimal pure strategy: delta = %.0f%%\n', delta(best_col_idx)*100);

tolerance = 1e-6;
if abs(V_pure - maximin_val) < tolerance
    fprintf('\n=== RESULT: Pure Saddle Point Exists ===\n');
    fprintf('Game value V = %.6f\n', V_pure);
else
    fprintf('\n=== RESULT: No Pure Saddle Point ===\n');
    fprintf('Proceeding to mixed strategy calculation...\n');

    f = [zeros(m, 1); 1];
    Aineq = [A', -ones(n, 1)];
    bineq = zeros(n, 1);
    Aeq = [ones(1, m), 0];
    beq = 1;
    lb = [zeros(m, 1); -Inf];

    options = optimoptions('linprog', 'Display', 'off');
    x = linprog(f, Aineq, bineq, Aeq, beq, lb, [], [], options);

    p_star = x(1:m);
    v_star = x(end);

    fprintf('\n--- Mixed Strategy Analysis ---\n');
    fprintf('Mixed security value: %.6f\n', v_star);

    tol = 1e-4;
    nonzero_idx = find(p_star > tol);
    fprintf('\nOptimal mixed strategy for Designer (nonzero probabilities):\n');
    fprintf('R_nom (  )   Probability\n');
    fprintf('--------   -----------\n');
    for k = 1:length(nonzero_idx)
        i = nonzero_idx(k);
        fprintf('   %.2f       %.4f\n', R_nom(i), p_star(i));
    end

    fprintf('\n--- Verification ---\n');
    expected_payoffs = A' * p_star;
    fprintf('Expected payoff against each delta:\n');
    for j = 1:min(5, n)
        fprintf('  delta = %5.1f%%: %.6f\n', delta(j)*100, expected_payoffs(j));
    end
    if n > 5
        fprintf('  ... (and %d more)\n', n-5);
    end

    max_expected = max(expected_payoffs);
    fprintf('\nMaximum expected payoff (worst-case): %.6f\n', max_expected);
    fprintf('This should equal mixed security value: %.6f\n', v_star);

```

```
    improvement = V_pure - v_star;
    improvement_percent = (improvement / V_pure) * 100;

    fprintf('\n--- Improvement Analysis ---\n');
    fprintf('Pure strategy value:    %.6f\n', V_pure);
    fprintf('Mixed strategy value:   %.6f\n', v_star);
    fprintf('Absolute improvement:   %.6f\n', improvement);
    fprintf('Relative improvement:   %.2f%%\n', improvement_percent);
end
```

Listing 2: MATLAB code for Exercise 9

Results

Running the MATLAB code produces the following output:

```
Matrix size: 11 (R_nom choices) × 21 (delta choices)

--- Pure Strategy Analysis ---
Designer (P1, minimizer):
  Security level (minimax): 0.111111
  Optimal pure strategy: R_nom = 1.00

Opponent (P2, maximizer):
  Security level (maximin): 0.025341
  Optimal pure strategy: delta = 8%

=== RESULT: No Pure Saddle Point ===
Proceeding to mixed strategy calculation...

--- Mixed Strategy Analysis ---
Mixed security value: 0.100000

Optimal mixed strategy for Designer (nonzero probabilities):
R_nom ()   Probability
--------   -----------
   1.00       0.7900
   1.05       0.2100

--- Verification ---
Expected payoff against each delta:
  delta = -10.0%: 0.100000
  delta = -9.0%: 0.087912
  delta = -8.0%: 0.076087
  delta = -7.0%: 0.064516
  delta = -6.0%: 0.053191
  ... (and 16 more)

Maximum expected payoff (worst-case): 0.100000
This should equal mixed security value: 0.100000

--- Improvement Analysis ---
Pure strategy value:    0.111111
Mixed strategy value:   0.100000
Absolute improvement:   0.011111
Relative improvement:   10.00%
```

Pure security strategy. The designer's pure security value is

$$V_{\text{pure}} = 0.111111,$$

which is achieved by choosing the nominal resistance

$$R_{\text{nom}} = 1.00\ \Omega.$$

The opponent's pure security value is

$$V_{\text{maximin}} = 0.025341,$$

achieved by choosing $\delta = 8\%$. Since $V_{\text{pure}} \neq V_{\text{maximin}}$, the game has no pure saddle-point equilibrium.

Mixed security strategy. Solving the corresponding linear program yields the mixed security value

$$V_{\text{mixed}} = 0.100000.$$

The optimal mixed security policy assigns positive probability to two nominal resistance values:

$$R_{\text{nom}} \in \{1.00\ \Omega,\ 1.05\ \Omega\},$$

with probabilities 0.7900 and 0.2100 respectively. All other nominal resistance values receive zero probability.

Verification. The verification step confirms the correctness of the solution: the maximum expected payoff against the optimal mixed strategy is exactly 0.100000, matching the computed mixed security value.

Improvement analysis. The mixed strategy provides an improvement of

$$\Delta V = V_{\text{pure}} - V_{\text{mixed}} = 0.011111 \quad \text{(approximately 10.00\%)}.$$

Analysis

The pure security analysis shows that selecting $R_{\text{nom}} = 1.00\ \Omega$ provides the best worst-case guarantee for the designer, with a maximum current deviation of 0.111111 regardless of the opponent's choice of δ. The opponent's optimal pure strategy is $\delta = 8\%$, which yields a minimum guaranteed gain of 0.025341.

Since $0.111111 \neq 0.025341$, the game has no pure saddle-point equilibrium. This gap between the minimax and maximin values indicates that randomization can potentially improve the designer's worst-case performance.

Indeed, by adopting a mixed security policy, the designer can reduce the guaranteed worst-case deviation to 0.100000, a significant improvement of 10.00% over the best pure strategy. The optimal mixed strategy combines $R_{\text{nom}} = 1.00\ \Omega$ (with 79.00% probability) and $R_{\text{nom}} = 1.05\ \Omega$ (with 21.00% probability).

The verification confirms that the worst-case scenario occurs when the opponent chooses $\delta = -10\%$, yielding an expected payoff of exactly 0.100000. For all other δ values, the expected payoff is lower, confirming that the mixed strategy successfully reduces the designer's maximum loss.

Discussion

The resistor design game demonstrates the practical value of mixed strategies in engineering design under uncertainty. The 10.00% improvement achieved through randomization is substantial, showing that mixed strategies can provide significantly better worst-case guarantees than pure strategies.

Key insights from this analysis:

1. **Optimal diversification**: The designer benefits from using a mixture of two closely spaced resistor values (1.00Ω and 1.05Ω) rather than committing to a single value.

2. **Robustness to worst-case variation**: The mixed strategy is particularly effective against the worst-case $\delta = -10\%$, which corresponds to the resistor being 10% smaller than nominal.

3. **Engineering implication**: In practice, this suggests that manufacturing resistors with a controlled distribution of nominal values (approximately 79% at 1.00Ω and 21% at 1.05Ω) would provide better worst-case performance than producing all resistors at exactly 1.00Ω.

4. **Theoretical significance**: The 10% improvement represents the value of randomization in this game. This gap between pure and mixed security values quantifies the benefit of being able to randomize design choices.

This exercise illustrates how game-theoretic concepts can inform robust engineering design decisions, providing quantitative methods for optimizing performance under uncertainty.